



LAB MANUAL

Name: Anam Shehzadi

Reg no.: 2023-BS-AI-048

Department: Artificial Intelligence

Semester: 5th

Subject: Natural language processing

Submitted to: Ms. Mirha Falak

INDEX

Sr No.	Lab Date	Lab Day	Task
1	01/09/2025	1	Tokenization
2	08/09/2025	2	Stop words
3			Custom stop words
4			Stemming
5	15/09/2025	3	Lemmatizer
6			POS tagging
7			Remove extra space
8			Emojis
9			Spell corrector
10	22/09/2025	4	Regular expressions
11	29/09/2025	5	Quiz
12	06/10/2025	6	Bag of words
13	13/10/2025	7	Word embedding
14	20/10/2025	8	N Gram
15	03/11/2025	9	Markov assumption
16	10/11/2025	10	Sentiment analysis
17	17/11/2025	11	Web scrapping
18	05/12/2025	12	Web Scrapping Project Implementation
19	12/12/2025	13	POS Tagging
20	19/12/2025	14	Training a Text Classifier (Spam/Ham or Sentiment Analysis)
21	26/12/2025	15	Implementation of HMM-based tagging

Lab 01

01/09/2025

Tokenization

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
nltk.download("punkt")
#nltk.download("punkt_tab") # Download the punkt_tab resource
text = "hello guys! My name is anummy."
sentences = sent_tokenize(text)
words = word_tokenize(text)
print("Original text:", text)
print("Sentence tokenize:", sentences)
print("word tokenize:", words)
```

```
Original text: hello guys! My name is anummy.
Sentence tokenize: ['hello guys!', 'My name is anummy.']
word tokenize: ['hello', 'guys', '!', 'My', 'name', 'is', 'anummy', '.']
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

Lab 02

08/09/2025

Stop words

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download("stopwords")
nltk.download("punkt_tab") # Download the missing resource
text = "this is a example of stopwords."
words = word_tokenize(text)
filtered_words = [word for word in words if word.lower() not in stopwords.words("english")] # Corrected 'words' to 'word'
print("Original:", words)
print("without stopwords:", filtered_words)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
```

```
Original: ['this', 'is', 'a', 'example', 'of', 'stopwords', '.']
without stopwords: ['example', 'stopwords', '.']
```

Custom Stop words

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download("stopwords")
nltk.download("punkt_tab") # Download the missing resource
text = "this is a example of stopwords."
words = word_tokenize(text)
custom_words = ["this", "is", "how"]
filtered_words = [words for words in words if words.lower() not in custom_words]
print("Original:", words)
print("without stopwords:", filtered_words)
```

```
Original: ['this', 'is', 'a', 'example', 'of', 'stopwords', '.']
without stopwords: ['a', 'example', 'of', 'stopwords', '.']
```

Stemming

```
from nltk.stem import PorterStemmer # library
ps = PorterStemmer() # function
print(ps.stem("playing"))
print(ps.stem("running"))
print(ps.stem("studies"))
print(ps.stem("cries"))
```

```
play
run
studi
cri
```

Lab 03**15/09/2025****Lemmatizer**

```

from nltk.stem import WordNetLemmatizer
import nltk
nltk.download("wordnet")
nltk.download("omw-1.4")
lemmatizer= WordNetLemmatizer()
print(lemmatizer.lemmatize("running", pos= "v"))
print(lemmatizer.lemmatize("better", pos= "a"))
print(lemmatizer.lemmatize("studies", pos= "n"))
print(lemmatizer.lemmatize("studies", pos= "v"))

```

```

run
good
study
study

```

POS tagging

```

import nltk
nltk.download("punkt")
nltk.download("averaged_perceptron_tagger_eng") # Download the correct resource
sentence = "anumy is pretty"
words = nltk.word_tokenize(sentence)
pos_tags = nltk.pos_tag(words)
print(pos_tags)

```

```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]   /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[('anumy', 'NN'), ('is', 'VBZ'), ('pretty', 'JJ')]

```

Text cleaning

```

import nltk
nltk.download("punkt")
nltk.download("averaged_perceptron_tagger_eng")
text = "    NLP is Great    "
print("Before:", repr(text))
clean_text = " ".join(text.split())
print("after:", repr(clean_text))

```

```

Before: '    NLP is Great    '
after: 'NLP is Great'

```

Handling emojis

```
!pip install emoji
import emoji
text = "I HATE NLP 🤖"
converted= emoji.demojize(text)
print("Before:", text)
print("After:", converted)
```

Collecting emoji

Downloading emoji-2.14.1-py3-none-any.whl.metadata (5.7 kB)

Downloading emoji-2.14.1-py3-none-any.whl (590 kB)

590.6/590.6 kB 5.4 MB/s eta 0:00:00

Installing collected packages: emoji

Successfully installed emoji-2.14.1

Before: I HATE NLP 🤖

After: I HATE NLP:loudly_crying_face: 🤖

Spell correcter

```
from textblob import TextBlob
text= "i liek ai, i haet engilsh"
corrected= TextBlob(text).correct()
print("Before:", text)
print("After:", corrected)
```

Before: i liek ai, i haet engilsh

After: i like ai, i hat english

Lab 04**22/09/2025****Regular expressions**

```
import re
nameage= ''' Janice is 22 and theon is 33 and anam is 44 and laiba is 55'''
nameage
```

```
' Janice is 22 and theon is 33 and anam is 44 and laiba is 55'
```

Extracting digits only:

```
import re
nameage= ''' Janice is 22 and theon is 33 and anam is 44 and laiba is 55'''
ages = re.findall(r'\d{1,3}', nameage)
ages
```

```
['22', '33', '44', '55']
```

```
import re
text= ''' Janice is 22 and theon is 133 and anam is 44 and laiba is 155'''
ages = re.findall(r'\d{1,3}', text)
print(ages)
```

```
['22', '133', '44', '155']
```

Alphabetic:

```
import re
nameage= ''' Janice is 22 and Theon is 33 and Anam is 44 and Laiba is 55'''
names = re.findall(r'[A-Z][a-z]*', nameage)
print(names)
```

```
['Janice', 'Theon', 'Anam', 'Laiba']
```

```
import re
text = "Happy Birthday google"
tokens = re.split(r"\s+", text)
print(tokens)
```

```
['Happy', 'Birthday', 'google']
```

Together all:

```
import re
text = "Anam is 20 and her pocket money is $5 and today is 05/05/20025."
ages = re.findall(r"\d{1,3}", text)
dates = re.findall(r"\d{2}/\d{2}/\d{4}", text)
prices = re.findall(r"\$\d+", text)
print("Ages=", ages)
print("dates=", dates)
print("price=", prices)
```

```
Ages= ['20', '5', '05', '05', '200', '25']
dates= ['05/05/2002']
price= ['$5']
```

Extracting cities:

```
text = "London is a very well known City of England and Anam love london"
names_places = re.findall(r"[A-Z][a-z]+", text)
print(names_places)
```

```
['London', 'City', 'England', 'Anam']
```

Mails extraction:

```
import re
text = "Contact us at anumy@gmail.com or at aimin@yahoo.org"
emails = re.findall(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", text)
print(emails)
```

```
['anumy@gmail.com', 'aimin@yahoo.org']
```

Hashtags and mentions:

```
import re
text = "Contact us at anumy@gmail.com or at aimin@yahoo.org #sunnyday #sad "
hashtags = re.findall(r"#\w+", text)
mentions = re.findall(r"@w+", text)
print("hashtags=", hashtags)
print("Mentions=", mentions)
```

```
hashtags= ['#sunnyday', '#sad']
Mentions= ['@gmail', '@yahoo']
```

Abbreviations:

```
import re
text= " i love studing books on AI, NLP and BIO."
abbreviations = re.findall(r"\b[A-Z]{2,}\b", text)
print(abbreviations)
```

```
['AI', 'NLP', 'BIO']
```


Lab 06

06/10/2025

Bag of words

```
from sklearn.feature_extraction.text import CountVectorizer
docs=[
    "i love machine learning",
    "machine learning is fun",
    "i like coding"
]
vectorizer=CountVectorizer()
x=vectorizer.fit_transform(docs)
print("vocabulary:",vectorizer.get_feature_names_out())
print("BOW Matrix:", x.toarray())
```

```
vocabulary: ['coding' 'fun' 'is' 'learning' 'like' 'love' 'machine']
BOW Matrix: [[0 0 0 1 0 1 1]
 [0 1 1 1 0 0 1]
 [1 0 0 0 1 0 0]]
```

Without using direct functions:

```
docs= [
    "i love machine learning",
    "machine learning is fun",
    "i love coding"
]
tokenized_docs=[doc.lower().split()for doc in docs]
vocab = sorted(set(word for doc in tokenized_docs for word in doc))
bow_matrix= []
for doc in tokenized_docs:
    vector=[doc.count(word)for word in vocab]
    bow_matrix.append(vector)
    print("vocabulary:", vocab)
    print ("bow Matrix:")
    for row in bow_matrix:
        print(row)
```

```
vocabulary: ['coding', 'fun', 'i', 'is', 'learning', 'love', 'machine']
bow Matrix:
[0, 0, 1, 0, 1, 1, 1]
vocabulary: ['coding', 'fun', 'i', 'is', 'learning', 'love', 'machine']
bow Matrix:
[0, 0, 1, 0, 1, 1, 1]
[0, 1, 0, 1, 1, 0, 1]
vocabulary: ['coding', 'fun', 'i', 'is', 'learning', 'love', 'machine']
bow Matrix:
[0, 0, 1, 0, 1, 1, 1]
[0, 1, 0, 1, 1, 0, 1]
[1, 0, 1, 0, 0, 1, 0]
```

Lab 07

13/10/2025

Word embedding

```
docs = [
    "artificial intelligence is changig the world",
    "deep learning improves computer vision systems",
    "natural language processing helps in translation",
    "AI and machine learning are transforming industries"
]
tokenized_docs=[doc.lower().split()for doc in docs]
vocab = sorted(set(word for doc in tokenized_docs for word in doc))
bow_matrix= []
for doc in tokenized_docs:
    vector=[doc.count(word)for word in vocab]
    bow_matrix.append(vector)
print("vocabulary:", vocab)
print ("bow Matrix:")
for row in bow_matrix:
    print(row)

vocabulary: ['ai', 'and', 'are', 'artificial', 'changig', 'computer', 'deep', 'helps', 'improves', 'in', 'industries', 'intelligence', 'is', 'language', 'learning', 'machine', 'natural', 'processing', 'systems', 'the', 'tran
bow Matrix:
[0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1]
vocabulary: ['ai', 'and', 'are', 'artificial', 'changig', 'computer', 'deep', 'helps', 'improves', 'in', 'industries', 'intelligence', 'is', 'language', 'learning', 'machine', 'natural', 'processing', 'systems', 'the', 'tran
bow Matrix:
[0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0]
vocabulary: ['ai', 'and', 'are', 'artificial', 'changig', 'computer', 'deep', 'helps', 'improves', 'in', 'industries', 'intelligence', 'is', 'language', 'learning', 'machine', 'natural', 'processing', 'systems', 'the', 'tran
bow Matrix:
[0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0]
vocabulary: ['ai', 'and', 'are', 'artificial', 'changig', 'computer', 'deep', 'helps', 'improves', 'in', 'industries', 'intelligence', 'is', 'language', 'learning', 'machine', 'natural', 'processing', 'systems', 'the', 'tran
bow Matrix:
[0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0]
[1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0]
```

```
from gensim.models import Word2Vec
doc = [
    ["Artificial", "intelligence", "is", "changing", "the", "world"],
    ["Deep", "learning", "improves", "computer", "vision", "systems"],
    ["NLP", "helps", "in", "translation"],
    ["AI", "and", "ML", "are", "transforming", "industries"]
]
cbow_model = Word2Vec(doc, vector_size=5, window=2, min_count=1, sg=0)
print("Vocabulary: ", cbow_model.wv.key_to_index)
print("Vector: ", cbow_model.wv["Artificial"])
print("words similar to translation: ", cbow_model.wv.most_similar("translation"))
skipgram_model = Word2Vec(doc, vector_size=5, window=2, min_count=1, sg=1)
print("Vocabulary: ", skipgram_model.wv.key_to_index)
print("Vector: ", skipgram_model.wv["Artificial"])
print("words similar to translation: ", skipgram_model.wv.most_similar("translation"))

Vocabulary: {'industries': 0, 'transforming': 1, 'intelligence': 2, 'is': 3, 'changing': 4, 'the': 5, 'world': 6, 'Deep': 7, 'learning': 8, 'improves': 9, 'computer': 10,
Vector: [-0.1233535  0.02211227  0.12094565 -0.05680101 -0.12347045]
words similar to translation: [('industries', 0.9513087272644043), ('Artificial', 0.4645385146141052), ('transforming', 0.36892473697662354), ('in', 0.36589574813842773),
Vocabulary: {'industries': 0, 'transforming': 1, 'intelligence': 2, 'is': 3, 'changing': 4, 'the': 5, 'world': 6, 'Deep': 7, 'learning': 8, 'improves': 9, 'computer': 10,
Vector: [-0.1233535  0.02211227  0.12094565 -0.05680101 -0.12347045]
words similar to translation: [('industries', 0.9513109922409058), ('Artificial', 0.46456998586654663), ('transforming', 0.36896198987960815), ('in', 0.36603060364723206),
```

Lab 08

20/10/2025

N-Gram

```

#include <iostream>
#include <sstream>
#include <vector>
using namespace std;
void generateNgrams(vector<string> words, int N) {
    cout << N << "-grams:\n";
    for (int i = 0; i <= words.size() - N; i++) {
        for (int j = 0; j < N; j++) {
            cout << words[i + j] << " ";
        }
        cout << endl;
    }
    cout << endl;
}

int main() {
    string text = "I love coding in C++";
    stringstream ss(text);
    string word;
    vector<string> words;

    // Split sentence into words
    while (ss >> word)
        words.push_back(word);

    generateNgrams(words, 1); // Unigrams
    generateNgrams(words, 2); // Bigrams
    generateNgrams(words, 3); // Trigrams
}

```

```

*** 1-grams:
I
love
coding
in
C++

2-grams:
I love
love coding
coding in
in C++

3-grams:
I love coding
love coding in
coding in C++

```

Lab 09

Markov Assumption

03/11/2025

```

text='i love machine learning. machine learning is fun. i love learning nlp.'
words=text.replace('.', '').split() # Added () to call the split method
from collections import defaultdict
markov_model=defaultdict(list)
for i in range(len(words)-1):
    current_word=words[i]
    next_word=words[i+1]
    markov_model[current_word].append(next_word)
print('word transition dictionary')
for key, value in markov_model.items(): # Added () to call the items method
    print(f'{key}->{value}')
import random
word=random.choice(words)
sentence=[word]
for i in range(10):
    # Check if the current word is in the markov model and has possible next words
    if word in markov_model and markov_model[word]:
        next_words=markov_model[word] # Corrected the dictionary name
        word=random.choice(next_words)
        sentence.append(word) # Corrected the append method call
    else:
        # If no next word is found, break the loop
        break

print('generated sentence')
print(' '.join(sentence)) # Corrected the print statement for the sentence

```

```

word transition dictionary
i->['love']
love->['machine', 'learning']
machine->['learning', 'learning']
learning->['machine', 'is', 'nlp']
is->['fun']
fun->['i']
i->['love']
generated sentence
love machine learning machine learning machine learning machine learning nlp

```

Lab 10

10/11/2025

Sentimental Analysis

```

from textblob import TextBlob
texts = [
    "i love this key, its amazing",
    "the service was terrible and slow",
    "the movie was okay, nothing special"
]
for text in texts:
    blob = TextBlob(text)
    sentiment = blob.sentiment.polarity
    if sentiment > 0:
        print(f"text : '{text}' -> Sentiment: positive")
    elif sentiment < 0:
        print(f"text : '{text}' -> Sentiment: negative")
    else:
        print(f"text : '{text}' -> Sentiment: neutral")

```

```

text : 'i love this key, its amazing' -> Sentiment: positive
text : 'the service was terrible and slow' -> Sentiment: negative
text : 'the movie was okay, nothing special' -> Sentiment: positive

```

```

import nltk
from nltk.sentiment import SentimentIntensityAnalyzer
nltk.download('vader_lexicon')
sia = SentimentIntensityAnalyzer()
texts = [
    "i love this key, its amazing",
    "the service was terrible and slow",
    "the movie was okay, nothing special",
    "i don't like this at all!"
]
for text in texts:
    scores = sia.polarity_scores(text)
    compound_score = scores['compound']
    if compound_score >= 0.05:
        sentiment = "positive"
    elif compound_score <= -0.05:
        sentiment = "negative"
    else:
        sentiment = "neutral"
    print(f"Text: '{text}' -> Sentiment: {sentiment}")

```

```

[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data] Package vader_lexicon is already up-to-date!

```

Lab 11**17/11/2025****Web scrapping****What is Web Scrapping?**

Web scraping means automatically collecting large amounts of data from websites.

- Most websites show data in HTML form, which is unstructured.
- After scraping, the data is converted into a structured format like:
 - CSV file
 - Excel sheet
 - Database

This structured data is then used for different applications like data analysis, machine learning, NLP, etc.

Parts of Web Scrapping

Web scraping has 2 main parts:

1. Crawler (Spider/Bot)

- A crawler is an AI-based program.
- It browses the web, moves from link to link.
- It finds the required pages that contain the data.

2. Scraper

- A scraper is a tool or code that extracts data from the web pages the crawler found.
- It reads the HTML, CSS, and even JavaScript-generated content (if advanced).
- Then it extracts only the required data.

Lab 12

05/12/2025

Web scrapping PROJECT Implementation

```

from bs4 import BeautifulSoup
import requests

url = 'https://en.wikipedia.org/wiki/List_of_largest_companies_in_the_United_States_by_revenue'

page = requests.get(url)

soup = BeautifulSoup(page.text, 'html')

print ( soup )

<html><body><p>Please set a user-agent and respect our robot policy https://w.wiki/4wJS. See also T400119.
</p></body></html>

soup.find('table')

soup.find_all('table')

[]

table = soup.find_all('table')

print(table)

[]

soup.find('table' , class_ = 'wikitable sortable')

world_titles = soup.find_all('th')

world_titles

[]

world_table_titles = [ title.text.strip() for title in world_titles]

print(world_table_titles)

[]

import pandas as pd

df = pd.DataFrame(columns = world_table_titles)

df

— 
  

column_data = soup.find_all('tr')

for row in column_data[1:]:
    row_data = row.find_all('td')
    individual_row_data = [ data.text.strip() for data in row_data ]
    print(individual_row_data)

    length = len(df)
    df.loc[length] = individual_row_data

df

— 
  

```

Lab 13
12/12/2025
POS Tagging

```
▶ import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng') # Corrected download for the specific resource
text = "Artificial intelligence is transforming the world"
words = nltk.word_tokenize(text)
pos_tags = nltk.pos_tag(words)
print("Words:", words)
print("POS tags:", pos_tags)
```

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]   /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
```

```
Words: ['Artificial', 'intelligence', 'is', 'transforming', 'the', 'world']
```

```
POS tags: [('Artificial', 'JJ'), ('intelligence', 'NN'), ('is', 'VBZ'), ('transforming', 'VBG'), ('the', 'DT'), ('world', 'NN')]
```


Lab 14

19/12/2025

Training a Text Classifier (Spam/Ham or Sentiment Analysis)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

# 1. Load CSV (make sure spam_dataset.csv is in the same folder or give full path)
data = pd.read_csv("spam_dataset.csv") # or "spam.csv" if that's your file name

# 2. Keep only required columns (defensive)
data = data[['label', 'message']]

# 3. Convert labels to numbers
data['label_num'] = data['label'].map({'ham':0, 'spam':1})

# 4. Split into train/test
X_train, X_test, y_train, y_test = train_test_split(
    data['message'], data['label_num'], test_size=0.2, random_state=42
)

# 5. Vectorize text -> TF-IDF
tfidf = TfidfVectorizer(stop_words='english')
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

# 6. Train Multinomial Naive Bayes
model = MultinomialNB()
model.fit(X_train_tfidf, y_train)

# 7. Predict on test set
y_pred = model.predict(X_test_tfidf)

# 8. Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# 9. Test with custom messages
test_message = ["You have won 5000 dollars!", "Hi, I will be late today"]
test_message_tfidf = tfidf.transform(test_message)
prediction = model.predict(test_message_tfidf)

print("\nTest messages:", test_message)
print("Predicted labels (1=spam, 0=ham):", prediction.tolist())
```

Lab 15

22/12/2025

Implementation of HMM-based tagging

```
# NLP Lab #15
# Implementation of HMM-based POS Tagging

# Step 1 – Import Libraries
import nltk
from nltk.corpus import brown
from nltk.tag import HiddenMarkovModelTagger
from sklearn.model_selection import train_test_split

# Step 2 – Download Required Corpora
nltk.download('brown')
nltk.download('universal_tagset')

# Step 3 – Prepare Training and Testing Data
# Load Brown corpus with universal tagset
tagged_sentences = brown.tagged_sents(tagset='universal')

# Split data: 80% training, 20% testing
train_data, test_data = train_test_split(
    tagged_sentences,
    test_size=0.2,
    random_state=42
)

# Step 4 – Train HMM Tagger
hmm_tagger = HiddenMarkovModelTagger.train(train_data)

# Step 5 – Evaluate Accuracy
accuracy = hmm_tagger.evaluate(test_data)
print("HMM POS Tagger Accuracy:", accuracy)

# Step 6 – Test HMM POS Tagger on Your Own Sentence
sentence = "The dog barks loudly"
tokens = sentence.split()

tagged_sentence = hmm_tagger.tag(tokens)
print("\nSentence:", sentence)
print("Tagged Output:", tagged_sentence)
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data] Package brown is already up-to-date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data] Package universal_tagset is already up-to-date!
/tmp/ipython-input-543979600.py:29: DeprecationWarning:
  Function evaluate() has been deprecated. Use accuracy(gold)
  instead.
  accuracy = hmm_tagger.evaluate(test_data)
HMM POS Tagger Accuracy: 0.9512519025382987
```

Sentence: The dog barks loudly

Tagged Output: [('The', 'DET'), ('dog', 'NOUN'), ('barks', '.'), ('loudly', 'ADV')]
